Chiffrer les données sous Linux (cryptsetup et LUKS)

Nom Prénom : **Dylan Spinelli**

BTS SIO SISR

Paris Ynov Campus

Sommaire

a- Introduction..... 1

b- Création d'une partition chiffrée avec cryptsetup..... 1

c- Ouvrons le conteneur chiffré..... 1

d- Création d'un système de fichiers ext4..... 1

i- Formater la partition..... 1

ii- Monter la partition..... 2

e- Ajouter une phrase de passe supplémentaire..... 2

f- Démonter le disque..... 2

g- Test de notre disque chiffré..... 3

a- Introduction





Dans cette partie, nous allons voir comment créer une partition chiffrée avec cryptsetup. Le but est de chiffrer une partition d'un disque interne ou externe

Tout d'abord il faut vérifier que le paquet cryptsetup est installée, si ce n'est pas le cas il faut bien sûre l'installer

```
root💀debian:/# apt install cryptsetup
```

b- Création d'une partition chiffrée avec cryptsetup

La syntaxe est la suivante

```
#cryptsetup --verify-passphrase --cipher= --key-size
```

Exemple

```
#cryptsetup --verify-passphrase --cipher serpent-cbc-essiv:sha256 --key-size 256 luksFormat /dev/sdb1
```

On fait simple on va laisser le système déterminer les options par défaut

On va créer le conteneur chiffré avec la commande suivante :

```
root💀storage:~# cryptsetup --verify-passphrase luksFormat /dev/sdb1
ATTENTION: Le périphérique /dev/sdb1 contient déjà une signature pour un superblock « ext4 ».

WARNING!
========
Cette action écrasera définitivement les données sur /dev/sdb1.

Are you sure? (Type uppercase yes): YES
Saisissez la phrase secrète pour /dev/sdb1 :
Vérifiez la phrase secrète :
```

Attention YES en majuscule

c- Ouvrons le conteneur chiffré

On va ouvrir le conteneur avec luksOpen. On va appeler notre conteneur **sdb1crypt** (on peut mettre le nom qu'on veut) :

```
root💀storage:~# cryptsetup luksOpen /dev/sdb1 sdb1crypt
Enter passphrase for /dev/sdb1:
```

d- Création d'un système de fichiers ext4

```
#fdisk -l
```

pour vérification

```
Disque /dev/mapper/sdb1crypt : 10 GiB, 10720641024 octets, 20938752 secteurs
Unités : secteur de 1 × 512 = 512 octets
Taille de secteur (logique / physique) : 512 octets / 512 octets
taille d'E/S (minimale / optimale) : 512 octets / 512 octets
```

i- Formater la partition

```
root💀storage:~# mkfs -t ext4 /dev/mapper/sdb1crypt
mke2fs 1.45.5 (07-Jan-2020)
Creating filesystem with 2617344 4k blocks and 655360 inodes
Filesystem UUID: 26ff15a3-6979-4683-b2dd-bd40d8405914
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done
```

ii- Monter la partition

```
root💀storage:~# mount /dev/mapper/sdb1crypt /mnt
```

On peut mettre des données dans notre partition sdb1crypt

```
root💀storage:/mnt# mkdir data
root💀storage:/mnt# touch file.txt
root💀storage:/mnt# ls
data  file.txt
root💀storage:/mnt#
```

e- **Ajouter une phrase de passe supplémentaire**

On peut rajouter jusqu'à 8 clés si on veut que chaque individue ait sa propre clé

```
root💀storage:~# cryptsetup luksAddKey /dev/sdb1
Enter any existing passphrase:
Enter new passphrase for key slot:
Verify passphrase:
```

On peut vérifier le nombre des clés avec la commande suivante

```
root💀storage:~# cryptsetup luksDump /dev/sdb1
```

Les deux clés créées sont encadrées en vert

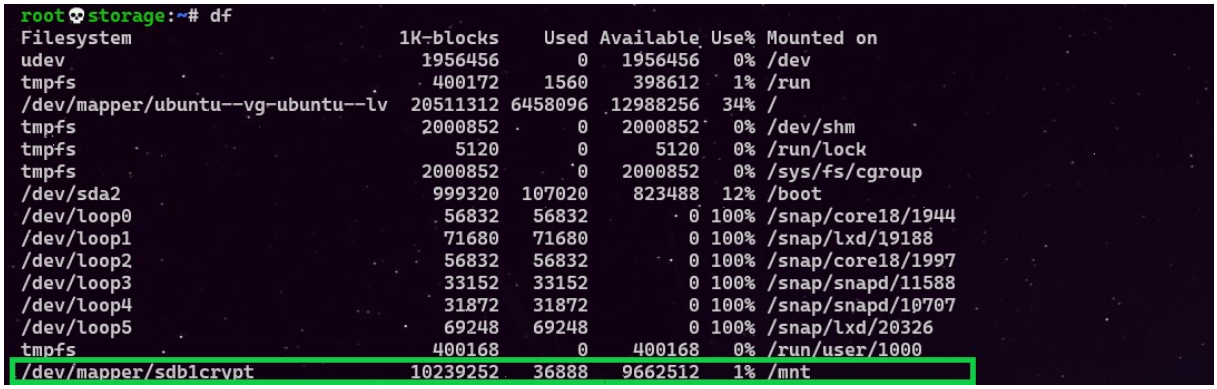
```
Keyslots:
  0: luks2
        Key:        512 bits
        Priority:   normal
        Cipher:     aes-xts-plain64
        Cipher key: 512 bits
        PBKDF:      argon2i
        Time cost:  4
        Memory:     311693
        Threads:    2
        Salt:       7b ee 30 1e f6 31 69 60 d6 24 fe 92 54 ed 5a 1d
                    aa d6 e9 c0 92 35 0d 27 f0 18 60 b0 e7 5f 09 c0
        AF stripes: 4000
        AF hash:    sha256
        Area offset:32768 [bytes]
        Area length:258048 [bytes]
        Digest ID:  0
  1: luks2
        Key:        512 bits
        Priority:   normal
        Cipher:     aes-xts-plain64
        Cipher key: 512 bits
        PBKDF:      argon2i
        Time cost:  4
        Memory:     339122
        Threads:    2
        Salt:       19 f2 0a a1 cb 69 12 d6 7c e9 2c 61 3e 14 df 40
                    d6 dd 76 c8 c8 72 42 bb ed 65 a5 0d a2 90 7c f3
        AF stripes: 4000
        AF hash:    sha256
        Area offset:290816 [bytes]
        Area length:258048 [bytes]
        Digest ID:  0
```

Pour supprimer une clé c'est simple on utilise la même commande sauf que on remplace **Add** par **Remove** et on entre le mot de passe à supprimer

```
root💀storage:~# cryptsetup luksRemoveKey /dev/sdb1
Enter passphrase to be deleted:
```

f- **Démonter le disque**

Maintenant on va démonter notre disque crypté sdb1 pour déterminer son point de montage on utilise la command df qui répertorie tous les points de montage



Notre point de montage est encadré en vert on utilise umount pour démonter le disque

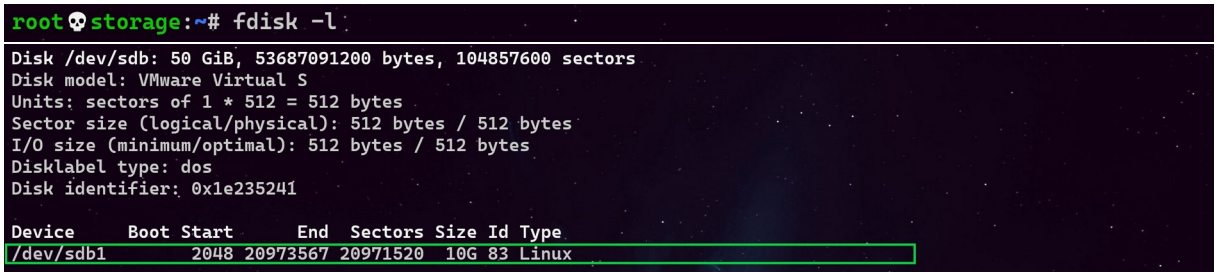


Maintenant il faut fermer notre conteneur chiffré



g- **Test de notre disque chiffré**

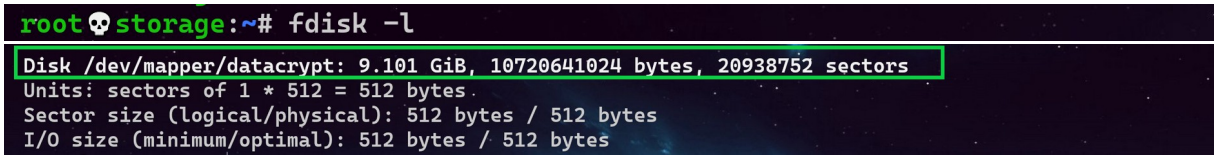
Il faut toujours faire un fdisk -l pour localiser notre disque car le nom peut changer en fonction d'un nouveau démarrage



Après on ouvre le conteneur chiffré on peut changer le nom de sdb1crypte à datacrypte
On rentre le mot de passe qu'on a entré précédemment



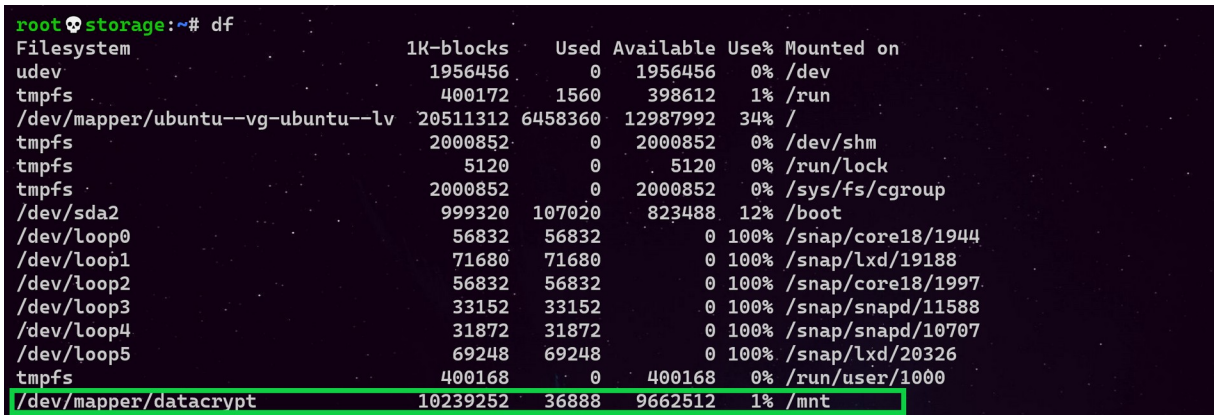
On vérifie avec fdisk -l pour identifier le volume à monter



On monte notre volume **/dev/mapper/datacrypt**



On vérifie avec df que le montage a réussi



Si on fait un ls on retrouve nos donnés